

Brac University

Department of Electrical & Electronic Engineering

Summer 2025

Course Number : EEE103L

Course Title : Computer Programming Laboratory

Section :06

Group No : 03



Lab Report

Experiment no: 06

Name of the experiment:

Pointers and Dynamic Memory Allocation

Prepared by:

Name: Souvik Barman Ratul ID: 24121205

All Group members:

<i>Sl.</i>	<i>ID</i>	<i>Name</i>
1.	24321069	Tahmid Jamil
2.	24121125	Shakila Hoque Nanjiba
3.	24121283	Sumaiya Fahmida Zerine
4.	24121205	Souvik Barman Ratul
5.	24121346	Mohammad Sadat Hossain Bhuiyan

Group contribution:

Writing : Souvik Barman Ratul

Coding : Shakila Hoque Nanjiba

Testing : Mohammad Sadat Hossain Bhuiyan

Problem Solving : Sumaiya Fahmida Zerine

Discussion : Tahmid Jamil

But each member of group 3 contributed to each segment by working together collaboratively.

Title and Objective:

- Title: Pointers and Dynamic Memory Allocation
- Objective: To learn how pointers are used in C for accessing data, arrays, and strings, and to understand how dynamic memory allocation functions (`malloc`, `calloc`, `realloc`, `free`) manage memory at runtime.

Theoretical background:

A **pointer** is a variable that stores the memory address of another variable.

- `&` gives the address.
- `*` gives the value at that address.

Pointers are useful for arrays, strings, function calls, and file handling.

Dynamic memory allocation (DMA): Memory can be allocated during program execution using:

- `malloc()` → allocates memory (garbage values).

- `calloc()` → allocates and initializes with 0.
- `realloc()` → changes memory size.
- `free()` → releases memory

Lab report assignments

a. Write a program to concatenate two strings using pointers.

Answer:

```
#include <stdio.h>
```

```
void concatenate(char *str1, char *str2) {
```

```
    while (*str1 != '\0')
```

```
    {
```

```
        str1++;
```

```
    }
```

```
    while (*str2 != '\0')
```

```
    {
```

```
        *str1 = *str2;
```

```
        str1++;
```

```
        str2++;
```

```
    }
```

```
    *str1 = '\0';
```

```
}
```

```
int main() {
```

Output

```
Concatenated string: Hello World
```

```
=== Code Execution Successful ===
```

```

char str1[100] = "Hello ";
char str2[] = "World";
concatenate(str1, str2);
printf("Concatenated string: %s\n", str1);
return 0;
}

```

b. Write a function to copy one array to another by using pointers.

Answer:

```

#include <stdio.h>

void copyArray(int *src, int *dest, int size) {
    int i;
    for (i = 0; i < size; i++) {
        *(dest + i) = *(src + i);    }
}

int main() {
    int arr1[5] = {10, 20, 30, 40, 50};
    int arr2[5];

    int i;

    copyArray(arr1, arr2, 5);
    printf("Copied array: ");
    for (i = 0; i < 5; i++) {
        printf("%d ", arr2[i]);
    }

    return 0;
}

```

Output

Copied array: 10 20 30 40 50

=== Code Execution Successful ===

c. Write a function to find out the dot multiplication of two vectors.

Answer:

```
#include <stdio.h>

int dotProduct(int *vec1, int *vec2, int size) {
    int sum = 0;
    for (int i = 0; i < size; i++) {
        sum += (*(vec1 + i)) * (*(vec2 + i));    }
    return sum;
}

int main() {
    int v1[5] = {1, 2, 3, 4, 5};
    int v2[5] = {2, 4, 6, 8, 10};
    int result;
    result = dotProduct(v1, v2, 5);
    printf("Dot product = %d\n", result);
    return 0;
}
```

Output

Dot product = 110

=== Code Execution Successful ===

Discussion:

In this experiment, we explored the use of pointers in C programming. We applied pointers to concatenate two strings, copy elements from one array to another, and calculate the dot product of two vectors. These tasks showed how pointers can directly access memory locations and make operations more efficient. We also discussed dynamic memory allocation functions like **malloc**, **calloc**, **realloc**, and **free**, which allow programs to manage memory during execution. Overall, this experiment helped us understand the practical importance of pointers for handling strings, arrays, and mathematical operations in an efficient way.

